UrbanFlow

Design Specification Document

Massa Aldeirani g00094979
Sara Zein g00095517
Batool Awad g00096367

American University of Sharjah
College of Engineering
Computer Science and Engineering

Version #: Design-mmddyyyy

Version History

Version	Author(s)	Change Description	Date	Notes

Table of Content

1 Overview

This section provides an overview of the design document. It outlines the purpose of the system, its overall structure, and the key components included in the design.

1.1 Software Context

The software is placed within an urban planning context where it supports district-level management and decision-making. It enables authorized users, including Administrators and Planners, to authenticate and manage districts, zones, and assets, and to define planning scenarios or events by applying operational changes to assets and infrastructure such as roads.

The system is intended for use by government planning authorities or authorized organizations to evaluate the impact of these events on district services and traffic conditions. It provides analysis through rule-based evaluation and computed indicators to support data-driven urban planning decisions.

1.2 Terminology & Definitions

District: A geographic area within the system that contains zones, assets, and infrastructure elements.

Zone: A defined subdivision within a district to which assets are assigned.

Asset: A physical community facility within a district, such as a park, gym, hospital, school, or recreational facility.

Planner: An authorized user who can create, modify, and evaluate planning scenarios and manage district data.

Administrator: An authorized user who manages user accounts, roles, and system data including districts, zones, and assets.

Planning Scenario: A set of defined operational changes applied to district assets or infrastructure elements within a district.

Operational Change: A modification applied within a scenario that specifies the affected entity, type of change, and modified attribute value.

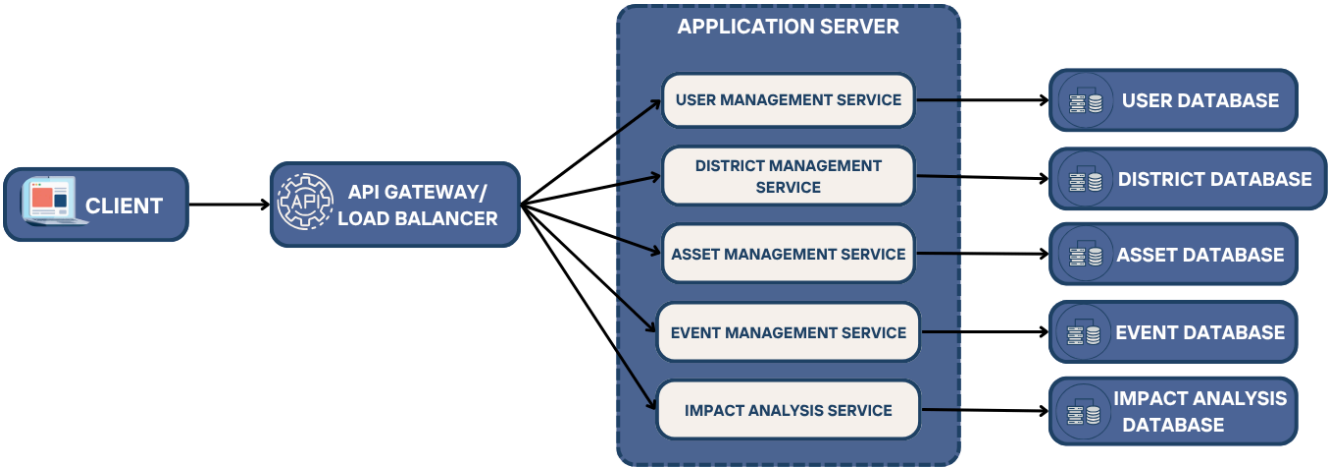
Impact Analysis: The system process of analyzing a planning scenario to determine its effect on district assets and roads.

Indicator: A rule-based computed value that estimates the impact of a scenario on asset capacity utilization and road traffic conditions.

2 System Architecture

The system adopts a cloud-based microservice architecture in which the main functions of the platform are distributed across multiple independent services and deployed on separate nodes. This architectural style supports the needs of the application by enabling secure user access, database per service management, impact analysis, and efficient communication between system components.

Several architectural styles were considered during the design of Urban Flow. A monolithic architecture was considered because it is simpler to implement in the early stages of development, but it was not selected because it becomes harder to scale and maintain as system complexity increases. A layered architecture was also considered since it provides a clear separation between presentation, business logic, and data layers, but it still keeps the system relatively tightly coupled. The selected microservice architecture was chosen because it provides better modularity, scalability, maintainability, and flexibility. Since Urban Flow includes multiple functional areas such as user, district management, asset management, event creation, and impact analysis, separating these functions into independent services is the most suitable architectural choice for the system.



2.1 Micro-service Architecture

The Urban Flow system is designed as a set of independent backend services that each handle a specific part of the platform. Instead of having one large application, the system is split into smaller services that communicate with each other through APIs. This makes the system easier to manage, update, and scale when needed.

At the front of the system, an API Gateway is used as the single entry point for all client requests. The web application does not communicate directly with individual services. Instead, every request goes through the gateway, which handles routing, basic security checks, and forwards the request to the correct service. This setup simplifies the frontend and keeps the internal structure of the system hidden.

The User Management Service manages user accounts, roles, and permissions. It is responsible for handling login and access control. It verifies user credentials and ensures that only authorized

users can access the system. This allows administrators to control who can access specific features of the system.

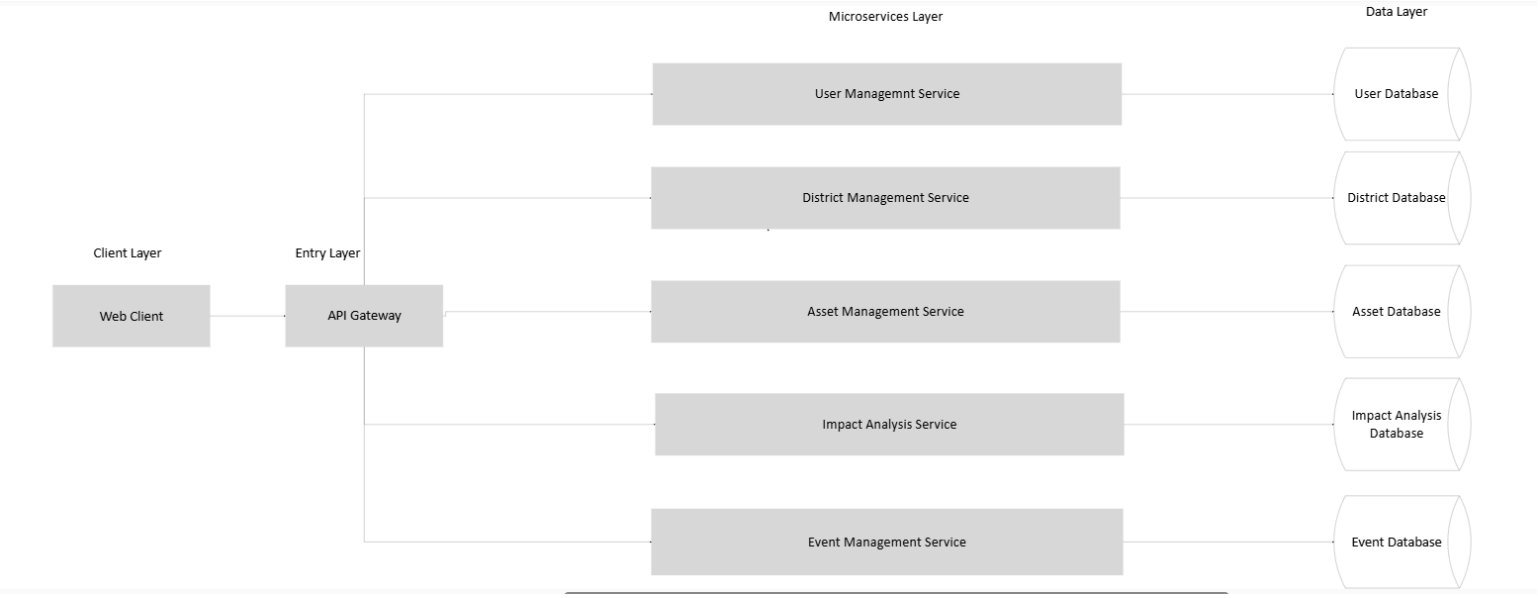
The District Management Service handles all operations related to districts and zones. It allows users to create, update, and organize districts, which act as the main working areas in the system. This service forms the base structure on which the rest of the system operates.

The Asset Management Service manages all facilities within a district, such as hospitals, schools, parks, and other resources. It stores important information like location, capacity, and status, which is later used in planning and evaluation.

The Event Management Service allows planners to define different planning events. Each event contains a set of operational changes that can affect assets within a district. These events are prepared and then sent for analysis.

The Impact Analysis Service performs the core analysis of the system. It takes an event and processes it to generate indicators related to asset usage and traffic conditions. This service handles the main computation logic and returns the results back to the user.

All services communicate using REST APIs over HTTP/HTTPS and exchange data in JSON format. It uses a database-per-service architecture, each microservice manages its own independent database to store its persistent data. This ensures better service isolation and allows each component to scale independently based on its workload. Data consistency across services is maintained through controlled API interactions rather than direct database access. Overall, this architecture improves modularity, maintainability, and flexibility for future system enhancements.



2.1.1 Web service interface description

Provide a detailed description of the exposed web service interfaces. This should include the interface name, parameters, return type/format, http method, and resource address.

The Urban Flow system has a set of RESTful web service interfaces to allow communication between the client application and the backend microservices. These interfaces are accessed through the API Gateway which routes each request to the correct internal service. The interfaces use standard HTTP methods such as GET, POST, PUT, and DELETE and the data format exchanged between client and server is JSON. The following interfaces represent the main exposed services of the system:

1. District Management Service

Interface Name: Create District

This interface creates a new district in the system.

- HTTP Method: POST
- Resource Address: /api/districts
- Parameters: districtName, location, description
- Return type: JSON object containing district details

Interface Name: Get All Districts

This interface retrieves all districts available in the platform.

- HTTP Method: GET
- Resource Address: /api/districts
- Parameters: None
- Return type: JSON array of district objects

Interface Name: Update District

This interface updates the data of an existing district.

- HTTP Method: PUT
- Resource Address: /api/districts/{districtId}
- Parameters: districtId, updated district data
- Return type: JSON object containing updated district information

Interface Name: Delete District

This interface deletes a district from the system.

- HTTP Method: DELETE
- Resource Address: /api/districts/{districtId}
- Parameters: districtId
- Return Type/Format: JSON confirmation message

Interface Name: Create Zone

This interface creates a new zone inside a selected district.

- HTTP Method: POST
- Resource Address: /api/districts/{districtId}/zones

- Parameters: districtId, zoneName, zoneDescription
- Return Type/Format: JSON object containing zone details

2. Asset and Service Management Service

Interface Name: Create Asset

This interface adds a new asset such as a school, hospital, park, or gym to the system.

- HTTP Method: POST
- Resource Address: /api/districts/{districtId}/assets
- Parameters: assetName, assetType, zoneId, location, capacity, status
- Return type: JSON object containing asset details

Interface Name: Get Assets by District

This interface retrieves all assets associated with a selected district.

- HTTP Method: GET
- Resource Address: /api/districts/{districtId}/assets
- Parameters: districtId
- Return type: JSON array of asset objects

Interface Name: Update Asset

Description: This interface updates the information of an existing asset.

- HTTP Method: PUT
- Resource Address: /api/districts/{districtId}/assets/{assetId}
- Parameters: assetId, updated asset data
- Return type: JSON object containing updated asset information

Interface Name: Delete Asset

This interface removes an asset from the system.

- HTTP Method: DELETE
- Resource Address: /api/districts/{districtId}/assets/{assetId}
- Parameters: assetId
- Return type: JSON confirmation message

3. Event Management Service

Interface Name: Create an Event

This interface allows a planner to create a new planning scenario.

- HTTP Method: POST
- Resource Address: /api/districts/{districtId}/events
- Parameters: districtId, plannerId, eventName, eventDescription
- Return type: JSON object containing event details

Interface Name: Add Operational Change

This interface adds an operational change to a selected event.

- HTTP Method: POST
- Resource Address: /api/districts/{districtId}/events/{eventId}/changes
- Parameters: scenarioId, affectedEntity, changeType, modifiedValue
- Return type: JSON object containing operational change details

Interface Name: Get Scenario Details

This interface retrieves the full details of a selected planning scenario.

- HTTP Method: GET
- Resource Address: /api/districts/{districtId}/events/{eventId}
- Parameters: eventId
- Return type: JSON object containing event data and operational changes

4. Impact Analysis Service

Interface Name: Impact-Analysis

This interface sends an event for processing and returns the impact analysis results.

- HTTP Method: POST
- Resource Address:/api/districts/{districtId}/events/{eventId}/impact-analysis
- Parameters: eventId
- Return type: JSON object containing impact analysis results and computed indicators

Interface Name: Get Impact Analysis Result

This interface retrieves the stored analysis result of a processed event.

- HTTP Method: GET
- Resource Address:/api/districts/{districtId}/events/{eventId}/impact-analysis/{resultId}
- Parameters: resultId
- Return type: JSON object containing impact indicators and analysis summary

5. Users and Authentication Service

Interface Name: User Login

This interface authenticates a user by verifying their credentials and returns their role and session details.

- HTTP Method: POST
- Resource Address: /auth/login
- Parameters: username, password
- Return type: JSON object containing authentication status, user role, and message

Interface Name: Create User

This interface creates a new user account in the system with specified credentials and role.

- HTTP Method: POST
- Resource Address: /users
- Parameters: username, password, role
- Return type: JSON object containing created user details (userId, username, role)

Interface Name: Get All Users

This interface retrieves a list of all registered users in the system.

- HTTP Method: GET
- Resource Address: /users
- Parameters: None
- Return type: JSON array containing user objects with ID, username, and role

Interface Name: Get User by ID

This interface retrieves details of a specific user by their ID.

- HTTP Method: GET
- Resource Address: /users/{id}
- Parameters: userId
- Return type: JSON object containing user details

Interface Name: Update User

This interface allows an administrator to update user account details or assigned roles.

- HTTP Method: PUT
- Resource Address: /api/users/{userId}
- Parameters: userId, updated user data
- Return type: JSON object containing updated user information

Interface Name: Delete User

This interface removes a user account from the system.

- HTTP Method: DELETE
- Resource Address: /api/users/{userId}
- Parameters: userId
- Return type: JSON confirmation message

These web service interfaces provide the main communication structure of Urban Flow. They support authentication, user and district management, asset handling, scenario definition, and scenario evaluation together. By organizing the interfaces around separate services, the system remains clear, modular, and easier to extend in future versions.

2.2 Node Architecture

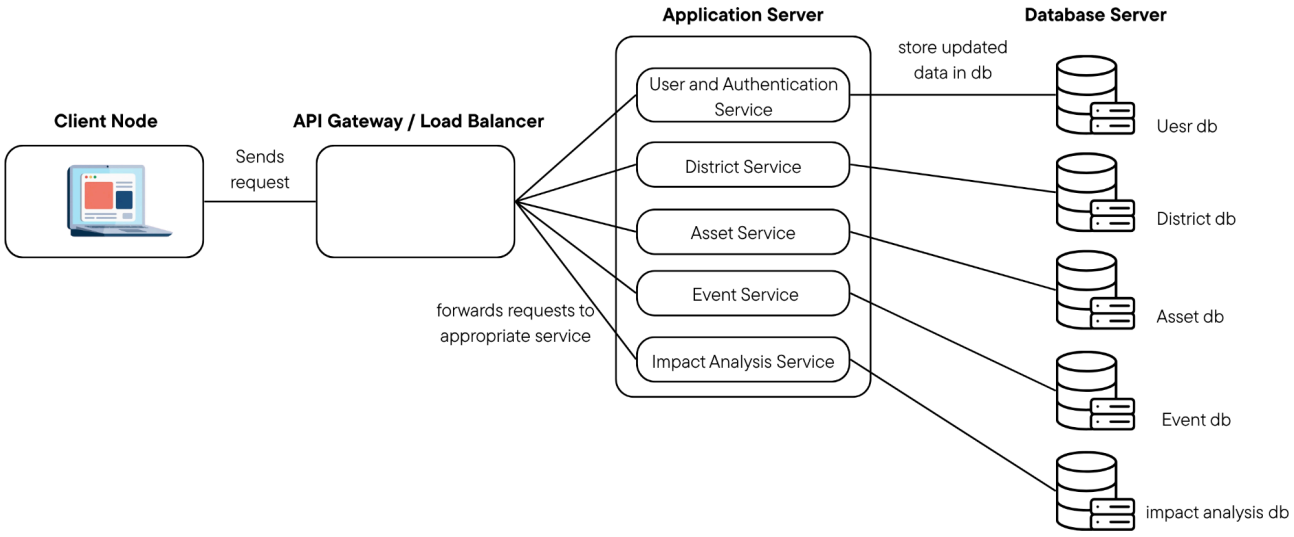
The node architecture describes how the Urban Flow system is physically deployed across different computing nodes and how these nodes communicate with each other. The system follows a distributed cloud-based architecture in which different components are deployed on separate nodes to improve scalability, performance, and reliability.

At the front layer, the Client Node represents the user interface accessed through a web browser. Users such as administrators and planners interact with the system through this node to perform operations including authentication, district and asset management, scenario creation, and viewing analysis results. All system requests originate from this node.

These requests are routed to the API Gateway / Load Balancer Node, which serves as the main entry point to the backend. It is responsible for directing incoming traffic to the appropriate services and distributing load across multiple instances, ensuring high availability and efficient handling of concurrent users.

The core system logic is handled by the Application Server Nodes, which host all microservices. Each microservice is independently deployed and responsible for a specific functional domain. These include the User Service, District Service, Asset Service, Event Service, Event Service, and Impact Analysis Service. The Impact Analysis Service, although computationally intensive, is implemented as a microservice and operates within the same layer as the other services. All services communicate using RESTful APIs over HTTP/HTTPS, enabling loose coupling and flexibility.

For data persistence, the system follows a database-per-service architecture, where each microservice is connected to its own dedicated database. This ensures data isolation, improves scalability, and reduces interdependencies between services. The databases store domain-specific data such as users, districts, assets, events, scenarios, and analysis results, and are accessed only through their corresponding services.



2.3 Data Architecture

The Urban Flow system follows a database-per-service architecture, where each microservice manages and maintains its own independent relational database instead of relying on a centralized database. This approach improves modularity, scalability, and fault isolation, as each service is responsible only for its own data. It also allows services to be developed, deployed, and scaled independently without impacting other parts of the system.

Each database stores structured data relevant to a specific functional domain of the system, while relationships between entities across different services are maintained using reference IDs and inter-service API communication, rather than direct foreign key constraints.

The User Management Service maintains a database containing the User and Role entities. The User entity stores account information such as username and password credentials, while the Role entity defines access privileges such as Administrator or Planner. A one-to-many relationship exists where one role can be assigned to multiple users.

The District Management Service manages a database containing the District and Zone entities. A District represents a geographic area within the platform and contains attributes such as name, location, and description. Each District can have multiple Zones, which act as subdivisions to organize different areas within the district. This establishes a one-to-many relationship between District and Zone.

The Asset Management Service maintains a database containing the Asset and Service entities. Assets represent physical facilities such as hospitals, parks, schools, and gyms, and include attributes such as location, capacity, and status. Services represent operational services available within a district. A one-to-many relationship exists between Service and Asset. Each Asset stores reference identifiers such as districtId and zoneId to link it logically to the District Management Service.

The Event Management Service manages a database containing the Event or Scenario and Operational Change entities. A Scenario represents a planning proposal created by a planner, and each Event can include multiple Operational Changes. These changes define modifications such as adjusting asset capacity or service availability. A one-to-many relationship exists between Event and Operational Change. Each Event stores reference identifiers such as plannerId and districtId to connect it to external services.

The Impact Analysis Service maintains a database containing the Evaluation Result and Indicator entities. Evaluation Results store the outcomes of event analysis, including metrics related to traffic conditions and asset utilization. Each Evaluation Result can include multiple Indicators, forming a one-to-many relationship. Each Evaluation Result references an Event using eventId.

Overall, this distributed data architecture removes tight coupling between services and replaces direct database relationships with loosely coupled service interactions, ensuring better scalability, maintainability, and flexibility for future system extensions.

3 Conclusions & Future Extensions

This document outlined the design of an urban planning system intended to support structured and data-driven decision-making within a district context. The system focuses on enabling controlled event definition and evaluation to assist planners in assessing potential outcomes before implementation.

Future extensions may include enhancing the accuracy of impact analysis through more advanced analysis models, incorporating real-time or external data sources, and improving the visualization of results. The system may also be extended to support additional types of infrastructure and more complex planning events.

---

4 Related Material

[1] requirement specification version filename.doc